CSc 453 — Assignment 5

Christian Collberg
University of Arizona, Department of Computer Science

March 20, 2026

1 Introduction

Your task is to write a semantic analyzer for the language Luca. You should build on the parser from assignment 4 that builds an abstract syntax tree. You semantic analyzer should

1. add syntehsized, inherited, and threaded attributes to the AST nodes,
2. walk the AST,
3. evaluate attributes, and
4. check context conditions over those attributes.

Your program should be named `luca_sem`. `luca_sem` reads a source program, does lexical, syntactic, and semantic analysis and writes semantic error messages to `standard error`. A semantically correct program will produce no output.

Your compiler should be invoked like this:

```
$ luca_sem prog1.luc
```

2 Theory Problems

In the following problems I want you to draw the environment in effect that a particular point (indicated by a •) in a given C program. The environment is a list of symbol tables, organized in such a way that if you search an environment E from beginning to end for a particular identifier I, *the first I you encounter is the correct one.*

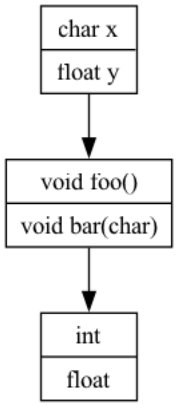
- Include *all* visible identifiers.
- Identifiers that belong to the same scope should be drawn in the same symbol table node.
- Include the type of all symbols.
- Assume that `int` and `float` are available in the standard scope.
- Use Graphviz to draw the environment. Enter your answer to problem i in the file `problem_`i`.gv`.
- You get 2 points per correct answer.

2.1 Example

Consider this C program `problem0.c`:

```
1  void foo() {
2  }
3
4  void bar(char x) {
5      float y;
6      •
7  }
8
9  int main() {
10     bar(99);
11 }
```

At the point indicated by •, the environment looks like this:



The corresponding Graphviz file looks like this:

```
digraph problem0 {
    standard [
        shape="record";
        label="{int | float}";
    ];
    sytab2 [
        shape="record";
        label="{void foo() | void bar(char)}";
    ];
    sytab1 [
        shape="record";
        label="{char x | float y}";
    ];
    sytab1 -> sytab2;
    sytab2 -> standard;
}
```

2.2 Problem 1

```
1  void foo(int x) {
2      int y;
3      {
4       int y;
5       •
6      };
7  }
8  int main() {
9      foo(99);
10 }
```

2.3 Problem 2

Note that this function can only be compiled with `gcc`, since nested functions are not part of the C standard.

```
1  void foo(char x) {
2      float y;
3      void bar(double x) {
4          short y;
5          void zap() {
6              •
7          }
8      }
9  }
10 int main() {
11     foo(99);
12 }
```

2.4 Problem 3

```
1  struct S {
2      int a;
3      float b;
4  };
5  void foo(char x) {
6      struct S z;
7      •
8  }
9  int main() {
10     foo(99);
11 }
```

3 Getting Started

This assignment can be done in teams of one or two.

To get started, follow these instructions:

1. Unzip `assignment-5.zip` which gives you these files:

```
   assignment-5/assignment-5.pdf
   assignment-5/firstname_lastname/sym/*.java
   assignment-5/firstname_lastname/sem/Semantics.java
   assignment-5/firstname_lastname/aux/Error.java
   assignment-5/firstname_lastname/tests/*.luc
   assignment-5/firstname_lastname/tests/*.out
   assignment-5/firstname_lastname/collaboration.json
   assignment-5/firstname_lastname/survey.json
   assignment-5/firstname_lastname/problem*.c
   assignment-5/firstname_lastname/problem_*.gv
survey
```

2. Then, assuming your name is Pat Jones, execute these commands:

```
> cd assignment-5
> mv firstname_lastname pat_jones
```

   If you are working in a group of two, pick one of your names.

3. Now, go ahead and solve the theory problems, entering your answers in the template files in `pat_jones/problem_*.gv`.

4. Symbol table and environment definitions have been provided for you in `assignment-5/firstname_lastname/sym/*.java`.

5. Copy your code from the previous assignments into `assignment-5/firstname_lastname/lexer/`, and `assignment-5/firstname_lastname/parser/`. `assignment-5/firstname_lastname/ast/`.

6. You will do the majority of your work in `assignment-5/firstname_lastname/sem/Semantics.java`. You will also have to add new attributes to the AST nodes in `assignment-5/firstname_lastname/ast/`.

4 Implementation Notes

- Your tree-walk evaluator should be programmed in an *applicative* style. I.e., all information should be passed around the tree using synthesized, inherited, and threaded attributes; there should be no global variables. In particular, you should use symbol tables and environments to represent scope information, as shown in lectures.
- The tree-walk evaluator should make exclusive use of recursion; iteration is not allowed. If you do make use of iteration and global data, points will be deducted.
- This assignment should be coded in Java.
- Make sure that your `Makefile` is working properly, and that `luca_sem` is called exactly as in the example above.
- You should not exit the semantic analyzer after the first error. Rather, you should generate as complete set of error messages as possible.

- You should avoid cascading error messages. Consider, for example, the expression $x + (y + 3.14)$ where x and y are both declared as integers. The subexpression $(y + 3.14)$ should generate a type error since integers can't be added to reals. However, $x + (\ldots)$ should not produce any "extra" messages as a result of the subexpression being erroneous.
- The Luca syntax is the same as in the previous assignment.

4.1 Symbols, Symbol tables, and Environments

In the `sym` subdirectory are classes for creating symbols (like `VariableSy` for variables and `ArrayType` for array declarations), symbol tables (sets of symbols), and environments (lists of symbol tables). You will use these during declaration analysis to construct symbol tables and environments that are then used during expression analysis.

These are the most important methods:

```
package sym;

public class SyTab implements Cloneable {
    public SyTab(sym.Symbol sy) {

    // Return a new symbol table consisting of the elements from S1 and S2.
    public SyTab merge (SyTab S2) {}

    // Return a fresh copy of SyTab to which Sy has been added.
    public SyTab insert (Symbol sy) {}

    // Return the symbol whose name is Name, if it exists in SyTab.
    // If not, return null
    public Symbol locateByName (String Name) {}
}
```

```
package sym;

public class Env {
    public Env(SyTab s) {}

   // Create a new environment by adding SyTab to the beginning of Env.
   public Env cons (SyTab s) {}

   // Create a new environment by adding SyTab to the end of Env.
   public Env snoc (SyTab s) {}

   public Symbol locateByName (String Name) {}
```

4.2 Error Messages

The error messages should be of this form:

```
<SEMANTIC_ERROR pos="3" message="Identifier expected" argument="X"/>
```

and should be written to standard output.

You can use these methods to generate error messages in the right format:

```
package aux;
public class Error {
    public static void Internal(String Proc, String Msg) {}
    public static int SemanticCount() {}
    public static void Sem (int Pos, String Msg) {}
    public static void SemId (int Pos, String Msg, String Id) {}
}
```

4.3 Debugging

Similar to previous assignments I would *strongly suggest* that you to also add code that

1. prints out the decorated AST in an XML format,
2. generates a Graphviz file that allows you to visualize the decorated tree, and
3. prints a trace of the execution of the recursive treewalker.

We will, however, not test any such debugging code.

5 Luca semantic rules

- Identifiers have to be declared before they are used.
- Identifiers cannot be redeclared in the same scope.
- There are four (incompatible) built-in types, `INTEGER`, `REAL`, `BOOLEAN` and `CHAR`.
- The identifiers `TRUE` and `FALSE` are predeclared in the language.
- Identifiers are case sensitive.
- Here are the type rules for Luca expressions:

Left	Operators	Right		Result
Int	'+', '−', '∗', '/', '%'	Int	⇒	Int
Real	'+', '−', '∗', '/'	Real	⇒	Real
Int	'<','<=', '=', '#', '>=', '>'	Int	⇒	Bool
Real	'<','<=', '=', '#', '>=', '>'	Real	⇒	Bool
Char	'<','<=', '=', '#', '>=', '>'	Char	⇒	Bool
Bool	'**AND**', '**OR**'	Bool	⇒	Bool
	'**NOT**'	Bool	⇒	Bool
	'−'	Int	⇒	Int
	'−'	Real	⇒	Real
	'**TRUNC**'	Real	⇒	Int
	'**FLOAT**'	Int	⇒	Real

- As seen from the table above, Luca does not allow *mixed arithmetic*, i.e. there is no *implicit conversion* of integers to reals in an expression. For example, if `I` is an integer and `R` is real, then `R:=I+R` is illegal.
- Luca instead supports two explicit conversion operators, `TRUNC` and `FLOAT`. `TRUNC R` returns the integer part of `R`, and `FLOAT I` returns a real number representation of `I`.
- Note that `%` (remainder) is not defined on real numbers.

- For a constant expression, division by 0 isn't allowed.
- Arithmetic operations on characters are not allowed.
- Variables are not allowed in constant expressions.
- Only reals, integers, and characters can be read or written.
- The left hand side of an assignment statement and the designator in a `READ` statement must be writeable (i.e. an L-value).
- EXIT statements can only occur within LOOP statements.
- A procedure's formal parameters and local declarations form one scope, which means that it is illegal for a procedure to have a formal parameter and a local variable of the same name.
- Parameters are passed by value unless the formal parameter has been declared **VAR**. Only L-valued expressions (such as '`A`' and '`A[5]`') can be passed to a **VAR** formal.
- The assignment `A:=B` is illegal if `A` or `B` are records or arrays.
- The types of the actual parameters to a procedure must match the formal parameters.
- An array can only be indexed by an integer expression.
- Given a record variable `R` of type `T`, in the field reference `R.f`, `f` must be a field declared in `T`.
- For student groups that have > 1 member, you also have to handle nested procedures.

6 Context Conditions

Below are the error conditions you need to check, organized by AST node. In some cases the order in which you check the conditions matters, particularly if you want to generate the same messages as I do! The reason is that we want to avoid cascading error messages — the test you perform first is more likely to produce an error message than one you perform later.

- At the end of semantic analysis I *sort* all my error messages lexicographically. If you do the same we're more likely to generate the same sequences of errors.

DECL:

An identifier can only be declared once in each scope. A procedure's formal parameters and local declarations form one scope. If `ID` is declared more than once, issue this error message:

```
<SEMANTIC_ERROR pos="..." message="Multiple declaration" argument="ID"/>
```

VARDECL,FIELDDECL,FORMALDECL:

1. The type name must be declared:

   ```
   <SEMANTIC_ERROR pos="..." message="Identifier not declared" argument="TypeName"/>
   ```

2. And, if the type name is declared, it has to be declared to be a type:

   ```
   <SEMANTIC_ERROR pos="..." message="Type identifier expected" argument="TypeName"/>
   ```

CONSTDECL:

1. The type name must be declared:

   `<SEMANTIC_ERROR pos="..." message="Identifier not declared" argument="TypeName"/>`

2. And, if the type name is declared, it has to be declared to be a type:

   `<SEMANTIC_ERROR pos="..." message="Type identifier expected" argument="TypeName"/>`

3. And, if it's declared a type, it has to be declared a *scalar* type (integer, character, real, boolean):

   `<SEMANTIC_ERROR pos="..." message="Scalar type expected"/>`

4. If the declared type is OK, you need to check that the expression is of the same type:

   `<SEMANTIC_ERROR pos="..." message="Wrong expression type"/>`

5. Regardless of the type checks above, the expression has to be constant-valued:

   `<SEMANTIC_ERROR pos="..." message="Constant expression expected"/>`

ARRAYDECL:

1. The type name must be declared:

   `<SEMANTIC_ERROR pos="..." message="Identifier not declared" argument="TypeName"/>`

2. And, if the type name is declared, it has to be declared to be a type:

   `<SEMANTIC_ERROR pos="..." message="Type identifier expected" argument="TypeName"/>`

3. The array size must be of type integer:

   `<SEMANTIC_ERROR pos="..." message="Integer expression expected"/>`

4. Regardless, of its type, the array size must be a constant expression:

   `<SEMANTIC_ERROR pos="..." message="Constant expression expected"/>`

ASSIGN:

1. The left hand and the right hand side must be of scalar (integer, real, char, boolean) type.

   `<SEMANTIC_ERROR pos="..." message="Scalar type expected"/>`

2. The left hand and the right hand side must be the same type.

   `<SEMANTIC_ERROR pos="..." message="Type missmatch in assignment statement"/>`

3. The left hand side must be a L-value, i.e. something you can assign to.

   `<SEMANTIC_ERROR pos="..." message="Can't assign to a constant"/>`

PROCCALL:

1. The designator must be a single declared identifier:

   `<SEMANTIC_ERROR pos="..." message="Identifier not declared"/>`

2. If the identifier is declared, it must be declared to be a procedure:

   `<SEMANTIC_ERROR pos="..." message="Procedure identifier expected"/>`

WRITE:

The expression must evaluate to an integer, real, character, or string.

```
<SEMANTIC_ERROR pos="..." message="INTEGER, REAL, CHAR, STRING type expected"/>
```

READ:

1. The designator must evaluate to an integer, real, or character:

   ```
   <SEMANTIC_ERROR pos="5" message="INTEGER, REAL, CHAR type expected"/>
   ```

2. The designator has to be an L-value (i.e. something you can assign to):

   ```
   <SEMANTIC_ERROR pos="..." message="Can't read to a constant"/>
   ```

WHILE,REPEAT,IF1,IF2:
The expression must be a boolean:

```
<SEMANTIC_ERROR pos="..." message="Boolean type expected"/>
```

EXIT:
`EXIT` must not occur outside of a loop:

```
<SEMANTIC_ERROR pos="..." message="EXIT only within LOOP"/>
```

ACTUAL:

1. There have to be the same number of actual and formal parameters:

   ```
   <SEMANTIC_ERROR pos="..." message="Too many actual parameters"/>
   <SEMANTIC_ERROR pos="..." message="Too few actual parameters"/>
   ```

2. Regardless, the actual parameter has to be assignable to the corresponding formal parameter.

   ```
   <SEMANTIC_ERROR pos="..." message="Actual/formal parameter type missmatch"/>
   ```

3. Regardless, if a formal parameter is declared to be a **VAR** parameter, then the corresponding actual has to be an L-value (cannot be a constant):

   ```
   <SEMANTIC_ERROR pos="..." message="VAR formal parameter requires variable actual"/>
   ```

VARREF:

1. The identifier has to be declared:

   ```
   <SEMANTIC_ERROR pos="..." message="Identifier not declared" argument="ID"/>
   ```

2. The identifier must be a formal parameter, a variable (global or local), a constant identifier, or a procedure:

   ```
   <SEMANTIC_ERROR pos="..." message="Variable expected"/>
   ```

INDEX:

1. The index expression must evaluate to an integer type:

   ```
   <SEMANTIC_ERROR pos="..." message="Integer type expected"/>
   ```

2. The designator must be of array type:

   ```
   <SEMANTIC_ERROR pos="..." message="Array variable expected"/>
   ```

FIELDREF:

1. The designator must be of record type:

   ```
   <SEMANTIC_ERROR pos="..." message="Record variable expected"/>
   ```

2. If the designator is or record type, then the field must be declared in the record:

   ```
   <SEMANTIC_ERROR pos="..." message="Field identifier not declared" argument="ID"/>
   ```

BINARY,UNARY:

The table in the previous section gives the semantic rules for expressions. For constant expressions, division by zero isn't allowed.

1. In arithmetic expressions, when a real or integer is expected, issue:

   ```
   <SEMANTIC_ERROR pos="..." message="Numeric type expected"/>
   ```

2. For $a\%b$, if a and b aren't integer types, issue

   ```
   <SEMANTIC_ERROR pos="..." message="Integer type expected"/>
   ```

3. For `AND`, `OR`, `NOT`, if the arguments aren't boolean types, issue

   ```
   <SEMANTIC_ERROR pos="..." message="Boolean type expected"/>
   ```

4. When the arguments to comparison operators (`#, <, >, <=, >=, =`) aren't integers, reals, booleans, or chars, issue

   ```
   <SEMANTIC_ERROR pos="..." message="Scalar or reference type expected"/>
   ```

   (Reference type refers to another version of Luca that also has pointer types.)

5. For `TRUNC` and `FLOAT`, respectively, when the arguments are of the wrong type, issue

   ```
   <SEMANTIC_ERROR pos="..." message="Real type expected"/>
   <SEMANTIC_ERROR pos="..." message="Integer type expected"/>
   ```

6. Otherwise, if the left and right hand sides are of different types, issue:

   ```
   <SEMANTIC_ERROR pos="..." message="Type missmatch"/>
   ```

7 Academic Integrity

- Don't show your code to anyone outside your team.
- Don't read anyone else's code.
- Don't discuss the details of your code with anyone.
- If you need help with the assignment see the TAs or the instructor.

7.1 Use of AI

You must write all code by hand without the use of any AI tools.

Q: Can I upload the assignment to an AI tool to explain it to me?

A: No. If you don't understand the assignment, ask the TAs.

Q: Can I use an AI tool to generate test cases?

A: No.

Q: In fact, are there *any* possible use cases for which it is OK for me to use AI tools in this assignment?

A: No, there are *no* such use cases.

Q: I will be using AI coding tools in my future job. So why can't I prepare for that by using such tools in this class?

A: This is a compiler class. The goal is to learn about compilers. If you want to learn about AI, take an AI class.

Q: But you will never be able to tell if my code was generated by an AI coding tool!

A: ROTFL!

8 Submission

- You should submit the assignment electronically to `d2l.arizona.edu`.
- You can work alone or in teams of 2.
- If you work in a team you should only submit one copy of the assignment.
- Your electronic submission *must* contain a working `Makefile`.

9 Grading

For the theory questions, you get 2 points per correct answer, for a total of 6 points.

You have been given 78 test cases. Each will give you one point if you get it right and 0 points if you get it wrong. No partial credits. We won't check for the correctness of line numbers.

There are an additional 2 test cases in `assignment-5/firstname_lastname/tests/NESTING` that have to be handled by groups with > 1 student. They are worth 2 points each.

There are additional, secret, test cases worth an additional 16 points for students in 1-person groups, 12 points for students in groups with > 1 student .

A Examples

```
PROGRAM T16;
VAR C:CHAR;
BEGIN
C := 'X' * 'Y';
END.
```

$\Longrightarrow$

```
<SEMANTIC_ERROR pos="4" message="Numeric␣type␣expected"/>
<SEMANTIC_ERROR pos="4" message="Numeric␣type␣expected"/>
```

```
PROGRAM T34;
PROCEDURE P(
   I:INTEGER;
   C:CHAR);
BEGIN
END;
BEGIN
P('x',3);
END.
```

$\Longrightarrow$

```
<SEMANTIC_ERROR pos="8" message="Actual/formal␣parameter␣type␣missmatch."/>
<SEMANTIC_ERROR pos="8" message="Actual/formal␣parameter␣type␣missmatch."/>
```

```
PROGRAM T394;
VAR x : REAL;
VAR y : REAL;
VAR z : REAL;
BEGIN
   x := y % z;
END.
```

$\Longrightarrow$

```
<SEMANTIC_ERROR pos="6" message="Integer␣type␣expected"/>
<SEMANTIC_ERROR pos="6" message="Integer␣type␣expected"/>
```